

Ingénierie de Prompts & iCloud Drive

Récapitulatif du projet — Claude Opus/Sonnet/Haiku

24 mars 2026

Ce projet regroupe (1) un guide complet d'ingénierie de prompts pour Claude, (2) un skill Claude Code prêt à l'emploi, et (3) deux scripts Python pour télécharger et uploader des documents vers iCloud Drive.

1. Guide — Ingénierie de Prompts pour Claude

Guide de référence basé sur la documentation officielle Anthropic (2026). Couvre tous les modèles : **Opus 4.6**, **Sonnet 4.6**, **Haiku 4.5**.

Chapitres

- 1. Fondamentaux — le principe du contrat, clarté avant longueur
- 2. Structure d'un prompt expert — ordre des éléments, squelettes
- 3. Techniques essentielles — XML tags, few-shot, permission d'incertitude
- 4. Contrôle du format de sortie — JSON, prose, supprimer les préambules
- 5. Raisonnement et Thinking — CoT, thinking adaptatif, auto-vérification
- 6. Utilisation des outils — appels parallèles, comportement proactif/conservateur
- 7. Systèmes agentiques — contexte long, actions réversibles, suivi d'état
- 8. Techniques avancées — meta-prompting, prompt chaining, RAG, constitutional
- 9. Anti-patterns à éviter
- 10. Checklists et templates

Règle d'or

Montre ton prompt à un collègue sans contexte.
S'il est confus, Claude le sera aussi.

Squelette universel

You are [RÔLE EN UNE PHRASE].

<context>[Contexte, audience, enjeux]</context>

<instructions>

1. [Règle principale]
2. [Contraintes]
3. [Comportement incertitude]

</instructions>

<output_format>[JSON / Markdown / prose / longueur]</output_format>

<examples>

<example>

<input>[Input type]</input>

```
<output>[Output idéal]</output>
</example>
</examples>
```

If you are not certain, say so. Do not guess.
Before finalizing, verify your response against the output format.

2. Techniques clés résumées

Anti-patterns à éviter

Anti-pattern	Conséquence	Correction
Instructions vagues	Comportement imprévisible	Critères de succès précis
Tâches multiples	Attention fragmentée	Chainer les appels API
Instructions négatives	Moins fiable	Formuler positivement
Documents après requête	-30 % de performance	Documents → requête
Pas d'exemples	Format aléatoire	3–5 exemples <exemple>
Pas de permission incertitude	Hallucinations confiantes	Clause d'incertitude
Pas d'auto-vérification	Erreurs non détectées	Étape de vérification finale

Paramètres API recommandés

Cas d'usage	Modèle	Effort	Thinking
Classification, extraction simple	Haiku 4.5	low	disabled
Rédaction, résumé, Q&A	Sonnet 4.6	medium	disabled
Code, analyse technique	Sonnet 4.6	medium	adaptive
Agent autonome multi-étapes	Opus 4.6	high	adaptive
Recherche longue, migration	Opus 4.6	max	adaptive

Checklist avant déploiement

- Critères de succès définis et mesurables
- Modèle optimal choisi (Haiku / Sonnet / Opus)
- Rôle défini en une phrase
- XML tags utilisés pour séparer les sections
- Documents placés avant la requête
- 3–5 exemples few-shot si format critique
- Format de sortie précisément décrit
- Permission d'incertitude ajoutée
- Auto-vérification finale demandée
- Testé sur 5+ cas représentatifs (dont cas limites)
- Version trackée (git / système de versioning)

3. Scripts Python — iCloud Drive

Deux scripts utilisent la bibliothèque **pyicloud** pour interagir avec iCloud Drive via l'API Apple. L'authentification 2FA est gérée automatiquement.

download_icloud.py — Téléchargement & organisation

Télécharge tous les fichiers iCloud Drive localement et recrée la même structure de dossiers directement dans iCloud Drive.

```
python download_icloud.py --username votre@apple.com
python download_icloud.py --username votre@apple.com --by-year
python download_icloud.py --username votre@apple.com --no-upload
```

Option	Description
--username / -u	Identifiant Apple (obligatoire)
--password / -p	Mot de passe (demandé si absent)
--output / -o	Dossier local (défaut : ~/iCloud_Backup)
--by-year	Ajouter sous-dossier par année de modification
--no-upload	Mode local uniquement, sans upload vers iCloud Drive
--verbose / -v	Logs détaillés

upload_docs_icloud.py — Upload ciblé

Uploading uniquement les fichiers du dossier **documents/** (DOCX, HTML) vers un dossier configurable dans iCloud Drive.

```
python upload_docs_icloud.py --username votre@apple.com
python upload_docs_icloud.py --username votre@apple.com --folder "Mes Prompts"
```

Option	Description
--username / -u	Identifiant Apple (obligatoire)
--password / -p	Mot de passe (demandé si absent)
--folder / -f	Nom du dossier iCloud Drive (défaut : Prompts_Expert)
--docs-dir	Dossier source local (défaut : ./documents/)

Structure des dossiers dans iCloud Drive

```
Documents_Organisés/
■■■ PDF/
■■■ Texte/
■■■ Tableurs/
■■■ Présentations/
■■■ Images/
■■■ Vidéos/
■■■ Audio/
■■■ Archives/
■■■ Code/
■■■ Autres/
```

Installation

```
pip install -r requirements.txt
```

Dépendances : pyicloud >= 1.0.0, click >= 8.0.0, tqdm >= 4.60.0

4. Fichiers du projet

Fichier	Type	Description
expert-prompt-guide.md	Markdown	Guide complet d'ingénierie de prompts
documents/expert-prompt-guide.docx	Word	Version Word du guide
documents/expert-prompt-guide.html	HTML	Version HTML du guide
documents/expert-prompt-skill.docx	Word	Skill Claude Code (Word)
documents/expert-prompt-skill.html	HTML	Skill Claude Code (HTML)
download_icloud.py	Python	Téléchargement & organisation iCloud Drive
upload_docs_icloud.py	Python	Upload ciblé vers iCloud Drive
convert_to_doc.py	Python	Conversion Markdown → DOCX/HTML
make_docx.py	Python	Génération DOCX avancée
make_pdf_recap.py	Python	Génération de ce PDF récapitulatif
requirements.txt	Config	Dépendances Python
README.md	Markdown	Documentation principale du projet